
```
(1.0, 60860.029867951016, 6.839403436723225, 7.620055170794695)
```

We interpret this output to mean that the best place to split the `hhmb` variable is at `1.0`; we split the survey respondents into people who live alone (one household member) and those who live with others (more than one household member). We can also see the average happiness levels for those two groups: about `6.84` and about `7.62`, respectively.

Choosing Splitting Variables

For any variable we choose in our data, we can find the optimal place to put our split point. However, remember that in a decision tree like the one in [Figure 9-1](#), we are not finding split points for only one variable. We split men from women, the obese from the non-obese, smokers from nonsmokers, and so on. A natural question is, how we should know which variable to split at each branching node? We could reorder the nodes in [Figure 9-1](#) so that we split by weight first and sex second, or sex only on the left branch or not at all. Deciding which variable to split at each branch point is a crucial part of generating an optimal decision tree, so we should write code for that part of the process.

We'll use the same principle we used to get optimal split points to decide the best split variable: the best way to split is the one that leads to the smallest error. In order to determine that, we need to iterate over each available variable and check whether splitting on that variable leads to the smallest error. We then determine which variable leads to the split with the lowest error. We can accomplish this by using [Listing 9-3](#).

```
def getsplit(data, variables, outcome_variable):
    best_var = ''
    lowest_error = float('inf')
    best_split = None
    predictedvalues = list(data.loc[:, outcome_variable])
    best_lowermean = -1
    best_highermean = -1
    for var in variables:
        allvalues = list(data.loc[:, var])
        splitted = get_splitpoint(allvalues, predictedvalues)

        if(splitted[1] < lowest_error):
```